

Solution Architecture

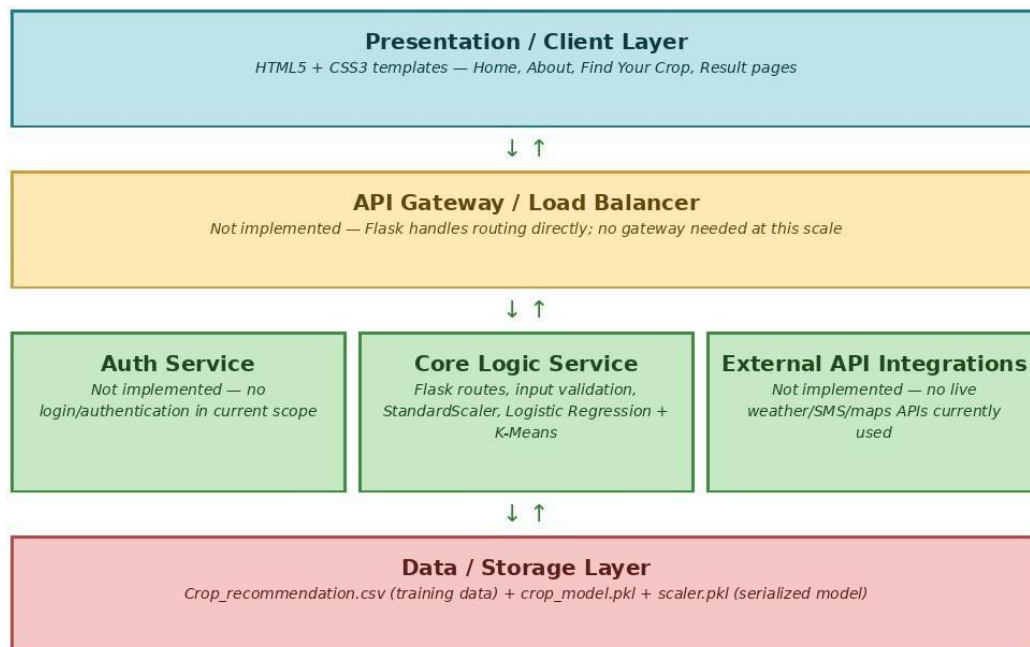
Date	03/07/2026
Team ID	SWTID-2026-5714
Project Name	OptiCrop: Smart Agriculture Production Optimization Engine
Maximum Marks	5 Marks

Solution Architecture Diagram:

This diagram maps OptiCrop's actual architecture onto the Presentation, Application, and Data layer structure.

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Renders the Home, About, Predict Crop, and Result pages. Collects the seven soil and climate input values from the user and displays the predicted crop recommendation.	HTML5, CSS3 (custom-styled, no JS framework)
API Gateway	<i>Not implemented. Flask's built-in routing handles all requests directly (/ , /about, /predict_page, /predict) — a separate gateway or load balancer is unnecessary at this project's scale (single instance, low traffic).</i>	<i>Not applicable</i>

Layers that are not part of the current implementation (API Gateway, Auth Service, External API Integrations) are explicitly marked as not implemented, rather than filled in with technologies that were not actually used.



Component Description Table



Core Logic / Microservices	A single Flask application (not split into microservices) that .validates the input, and executes the trained Random Forest modelto predict the recommended crop. K-Means clustering is used separately for exploratory analysis, not live prediction.	Python, Flask, Scikit-learn (RandomForestClassifier), NumPy
Database	Stores the training dataset and the serialized trained model. No database server is used — the CSV is read at training time, and the model/scaler are loaded into memory from disk when the Flask app starts.	CSV file (Crop_recommendation.csv), Pickle (crop_model.pkl, scaler.pkl)

Note: Auth Service and External API Integrations shown in the diagram template are not part of the current build — there is no user login and no external (weather, payment, email, etc.) API calls in this version of OptiCrop.